CS 301
High-Performance Computing

Assignment 04
Particle Interpolation and Mover Performance Analysis

Khushi Pandya (202301406)
Aalok Thakkar (202301429)

March 11, 2026

Contents

1 Introduction

The objective of this assignment is to analyze the performance of a particle interpolation algorithm and a particle movement phase in a two–dimensional computational domain. The experiment evaluates how execution time scales with increasing particle count and different grid configurations.

In addition to interpolation, a new operation called the **Mover** is introduced, where particles are randomly displaced inside the computational domain. The performance of the mover phase is analyzed in both serial and parallel implementations using OpenMP.

All experiments are executed on both a Lab PC and an HPC cluster to compare their performance characteristics.

2 Problem Description

The computational domain is defined as

$$0 \leq x \leq 1, \quad 0 \leq y \leq 1$$

Particles are randomly distributed within this domain. The goal of the interpolation algorithm is to map particle values onto a structured mesh grid.

The grid resolution is

$$N_x \times N_y$$

Each particle contributes to the surrounding four grid nodes using bilinear interpolation weights.

The grid spacing is defined as

$$\Delta x = \frac{1}{N_x}, \qquad \Delta y = \frac{1}{N_y}$$

3 Implementation Overview

3.1 Cell Identification

For a particle located at position (x, y), the grid cell index is computed as

$$i = \left\lfloor \frac{x}{\Delta x} \right\rfloor, \qquad j = \left\lfloor \frac{y}{\Delta y} \right\rfloor$$

The normalized distances inside the cell are

$$d_x = \frac{x - X_i}{\Delta x}, \qquad d_y = \frac{y - Y_j}{\Delta y}$$

3.2 Weight Calculation

Each particle contributes to four neighboring grid points

$$w_{i,j} = (1 - d_x)(1 - d_y)$$

$$w_{i+1,j} = d_x(1 - d_y)$$

$$w_{i,j+1} = (1 - d_x)d_y$$

$$w_{i+1,j+1} = d_x d_y$$

The weights satisfy

$$\sum w = 1$$

4 Experiment 01: Scaling with Number of Particles

This experiment analyzes how interpolation time scales with increasing particle counts.

Three grid configurations were tested:

- Configuration 1: $N_x = 250$, $N_y = 100$
- Configuration 2: $N_x = 500$, $N_y = 200$
- Configuration 3: $N_x = 1000$, $N_y = 400$

Particle counts tested:

$$10^2, 10^4, 10^6, 10^8, 10^9$$

Interpolation was executed for $Maxiter = 10$ iterations.

4.1 Results

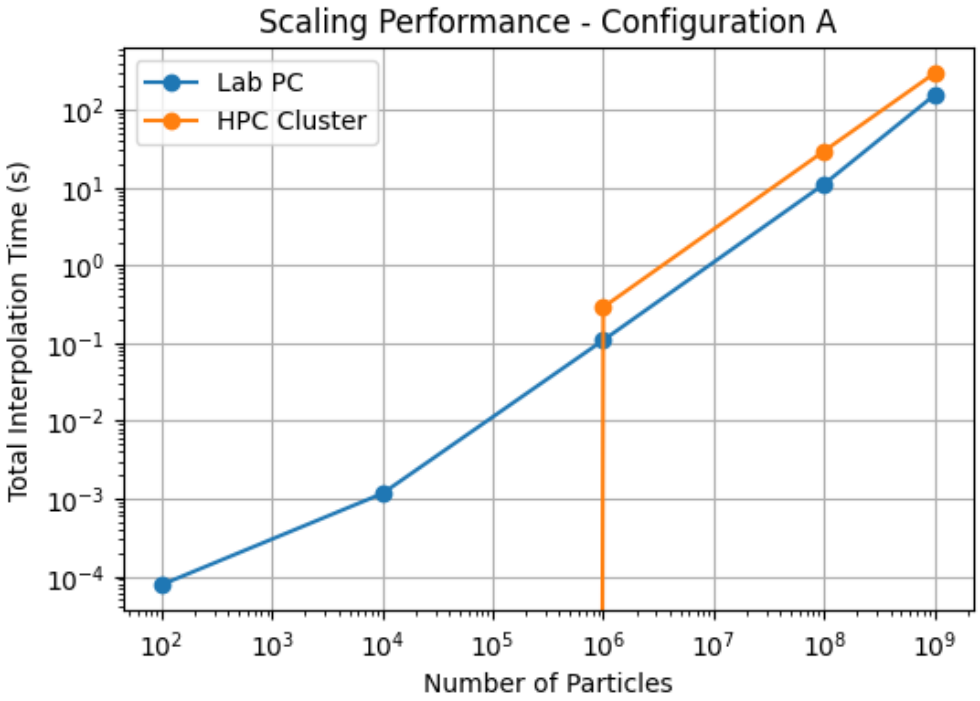

Figure 1: Interpolation time vs particle count for configuration 1

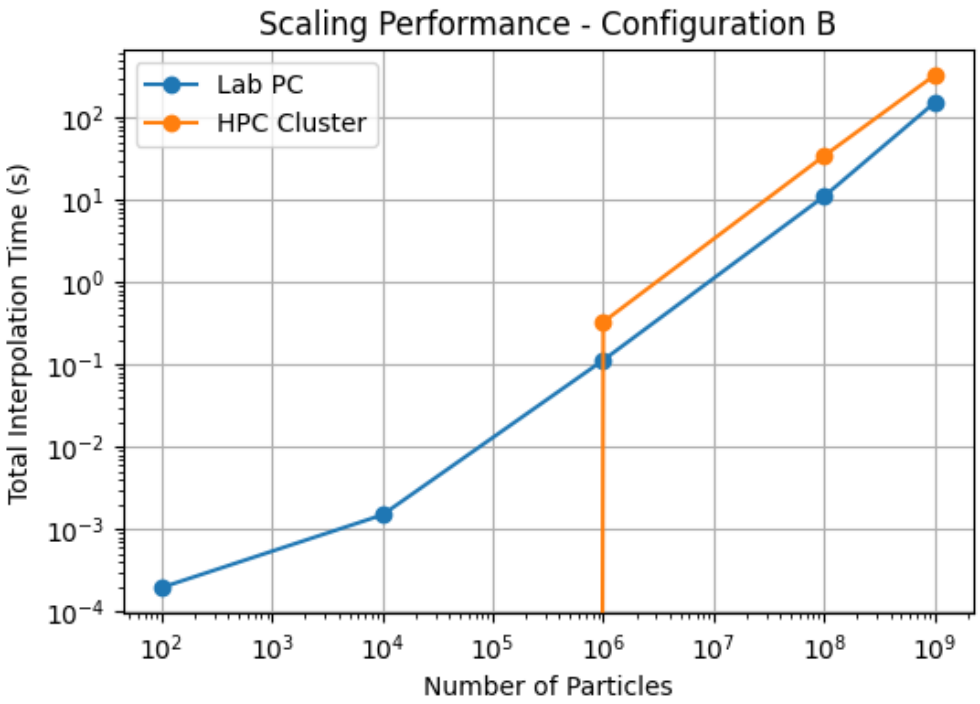

Figure 2: Interpolation time vs particle count for configuration 2

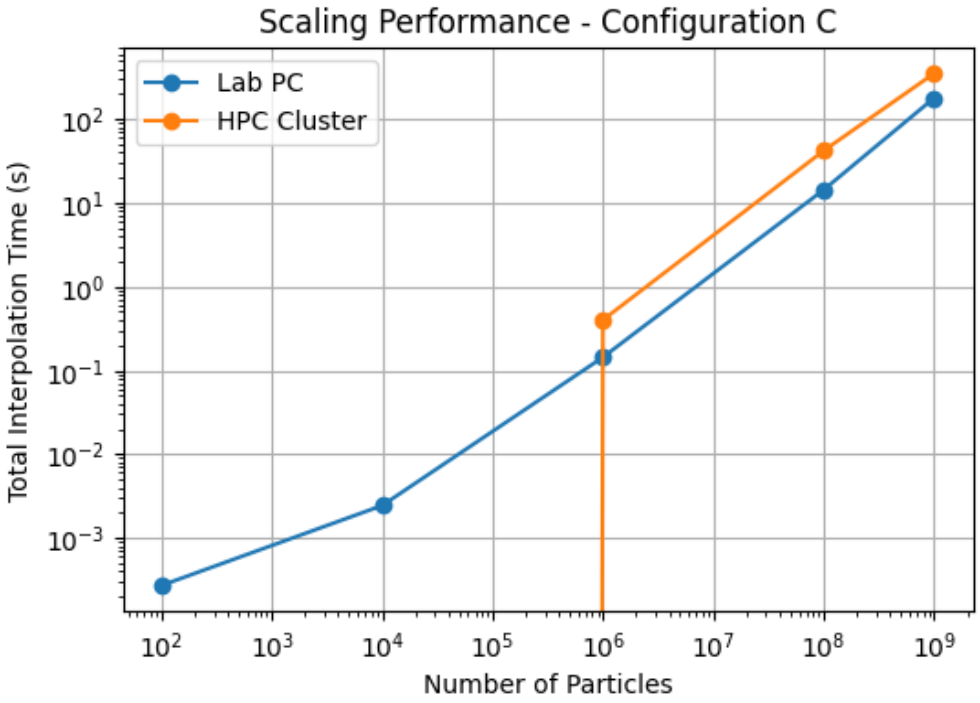

Figure 3: Interpolation time vs particle count for configuration 3

5 Experiment 02: Consistency Across Configurations

In this experiment, the particle count was fixed at

$$10^8$$

Interpolation was executed for 10 iterations without reinitializing particles.

5.1 Lab PC

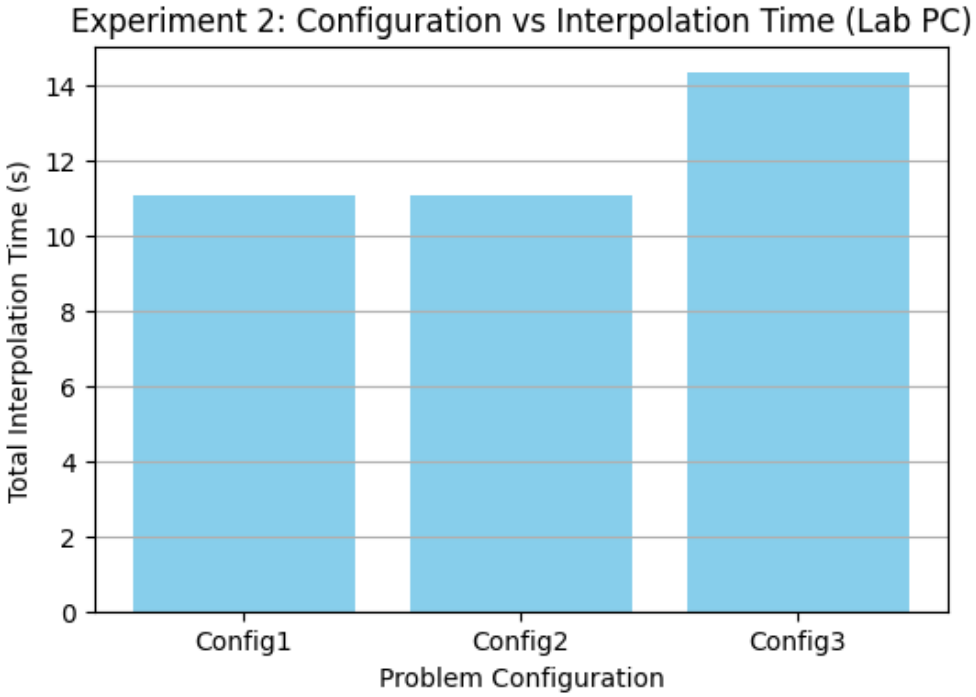

Figure 4: Interpolation time vs problem configuration (Lab PC)

5.2 HPC Cluster

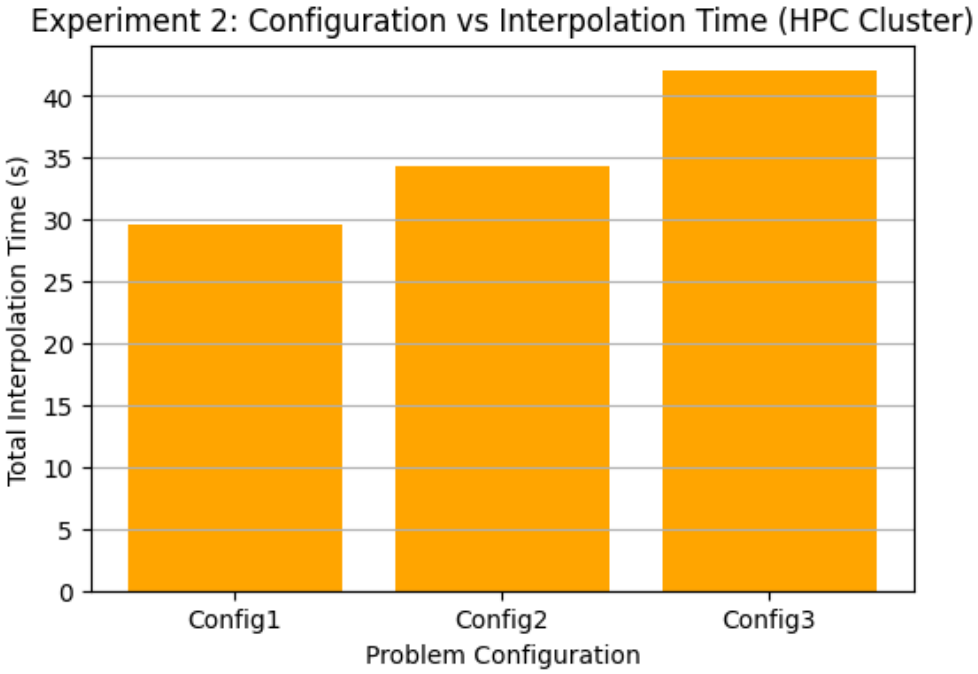

Figure 5: Interpolation time vs problem configuration (HPC cluster)

6 Experiment 03: Mover Operation and Parallelization

After interpolation, particles are randomly displaced within the domain

$$-\Delta x \leq \delta x \leq \Delta x$$

$$-\Delta y \leq \delta y \leq \Delta y$$

Particles must remain within

$$0 \leq x, y \leq 1$$

The mover operation was parallelized using OpenMP with four threads.

6.1 Iteration Timing

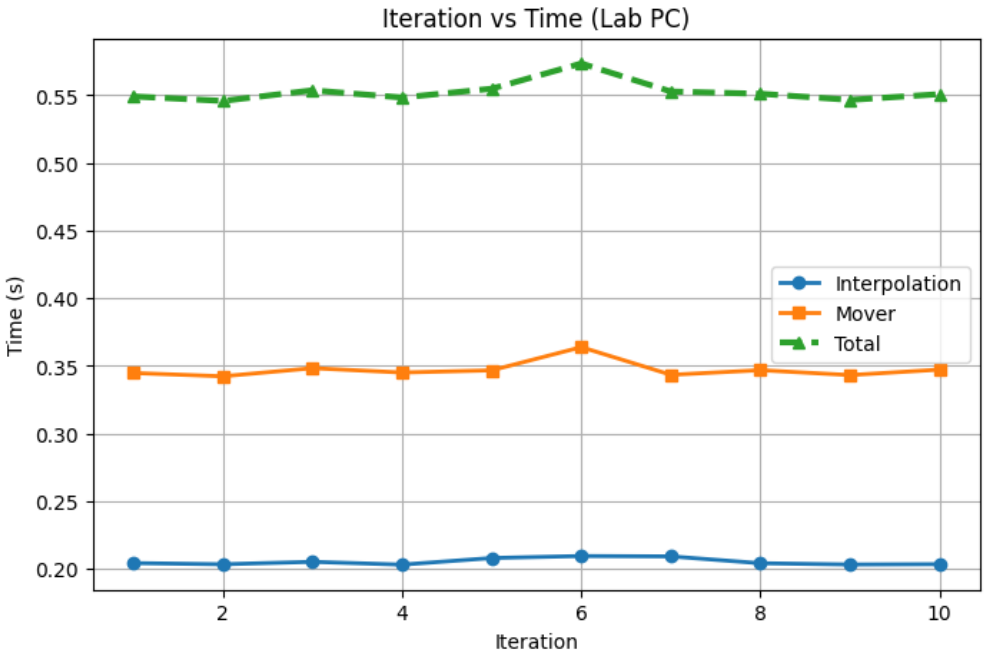

Figure 6: Interpolation time, mover time and total time across iterations (Lab PC)

Figure 7: Interpolation time, mover time and total time across iterations (HPC cluster)

6.2 Serial vs Parallel Mover

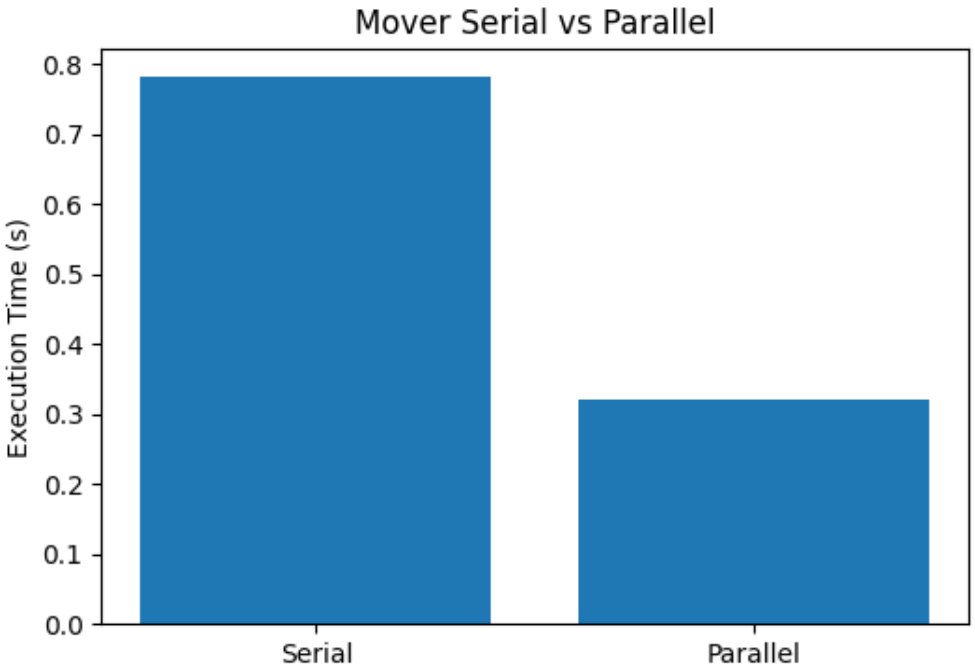

Figure 8: Serial vs Parallel mover execution time

6.3 Speedup

Speedup is calculated as

$$S = \frac{T_{serial}}{T_{parallel}}$$

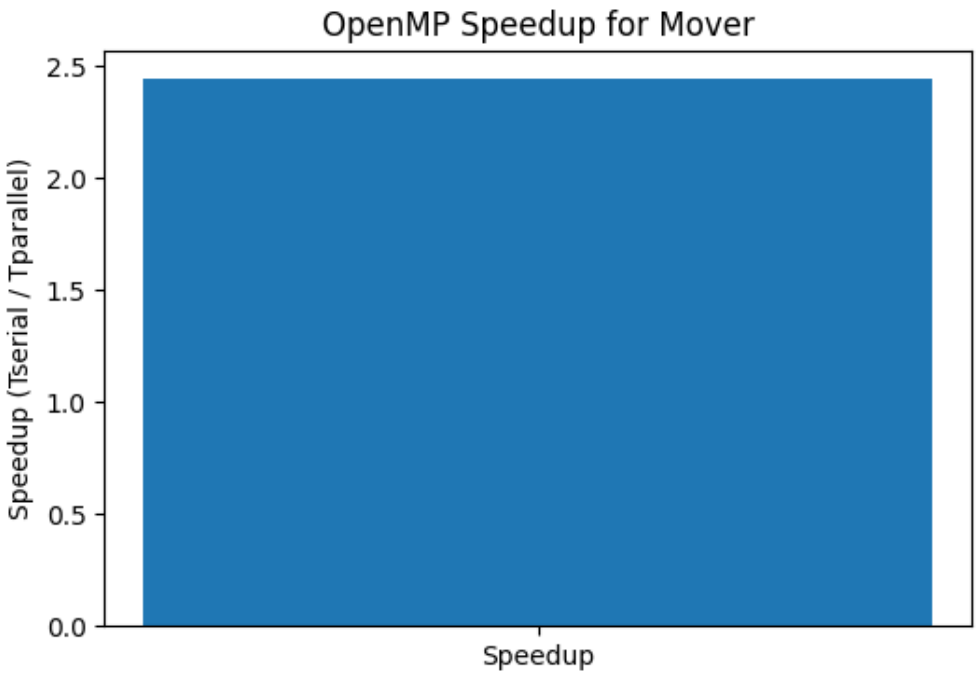

Figure 9: Speedup achieved using OpenMP parallelization

7 Performance Comparison

The HPC cluster generally demonstrates better performance due to higher computational resources and improved memory bandwidth.

However, performance differences may also arise due to differences in CPU architecture, cache hierarchy, and clock speeds.

8 Analysis

The interpolation algorithm shows near-linear scaling with the number of particles. As particle count increases, the total execution time increases proportionally.

The mover operation benefits significantly from parallelization because each particle update is independent.

Parallelizing interpolation is more challenging since multiple particles may update the same grid node simultaneously, causing potential race conditions.

Possible solutions include:

- Atomic updates
- Thread-local grids with reduction
- Spatial partitioning of particles

8.1 Lab PC

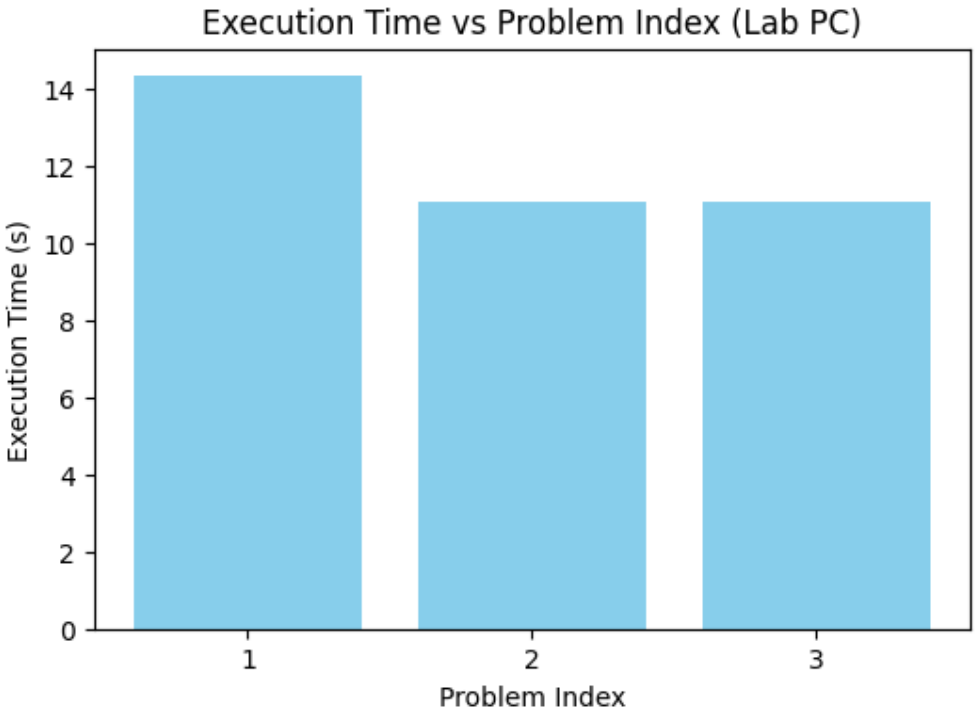

Figure 10: Execution time vs problem index for Lab PC.

8.2 HPC Cluster

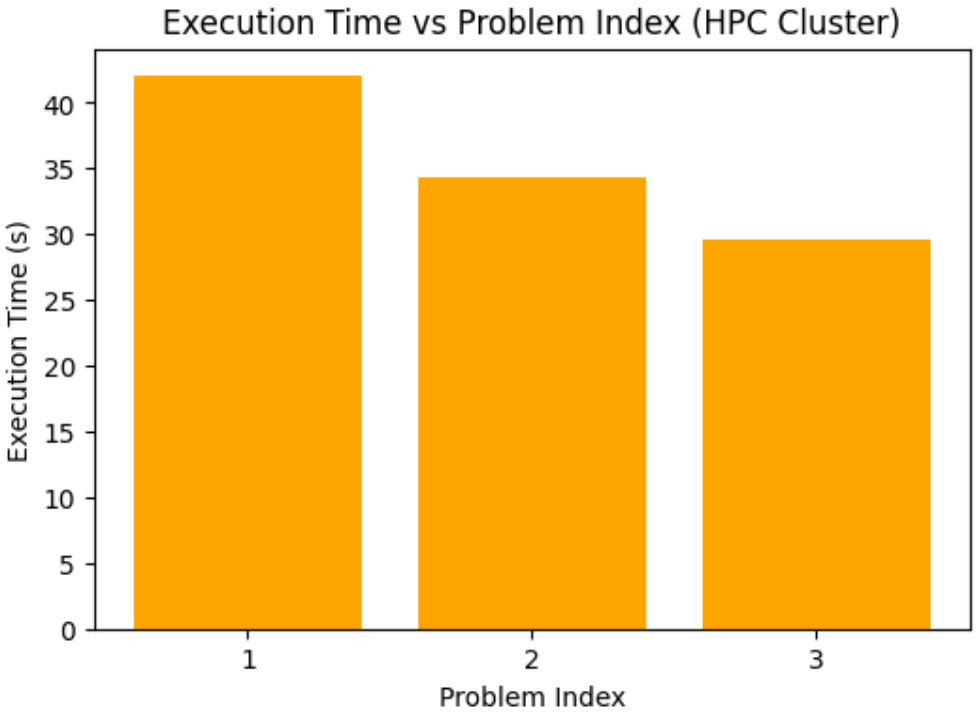

Figure 11: Execution time vs problem index for HPC cluster.

9 Conclusion

This assignment analyzed the performance of particle interpolation and mover operations in a two-dimensional domain.

The results show that interpolation performance scales with the number of particles and grid size. The mover phase can be efficiently parallelized using OpenMP due to the independence of particle updates.

Overall, the experiments demonstrate the importance of memory access patterns and parallel execution in high-performance computing applications.